

**SIMATS ENGINEERING**  
**CO-1 ASSESSMENT TOOL 3 - Low (Pseudo-code Writing)**

**COURSE CODE: CSA09**

**COURSE NAME: Programming in Java**

**COURSE OUTCOME COVERED**

**CO1:** *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

Assessment Tool Weightage: 40%

**QUESTIONS: 10**

**Total Marks: 100**

ANNEXURE A

**PSEUDO-CODE WRITING**

***Code of Conduct:***

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

ANNEXURE

**PSEUDO-CODE WRITING**

**1. Student Management System [BL3 – Apply]**

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details

- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

**OOP Concepts:** *Encapsulation | Data Hiding | Classes & Objects*

---

## 2. Library Book Management [BL3 – Apply]

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

**OOP Concepts:** *Encapsulation | Getters & Setters | Objects*

---

## 3. Employee Salary Calculation [BL3 – Apply]

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)

- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.

**OOP Concepts:** *Encapsulation | Data Hiding | Methods*

---

## 4. Shape Area Calculator using Polymorphism [BL3 – Apply]

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

**OOP Concepts:** *Inheritance | Polymorphism | Method Overriding*

---

## 5. Vehicle Registration System [BL3 – Apply]

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

**OOP Concepts:** *Inheritance | Method Overriding | Subclass*

## 6. Bank Account Hierarchy [BL4 – Analyze]

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

**OOP Concepts:** *Inheritance | Method Overriding | Polymorphism*

---

## 7. Hospital Patient Record System [BL4 – Analyze]

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences.

**OOP Concepts:** *Encapsulation | Inheritance | Polymorphism*

---

## 8. E-Commerce Product Catalog [BL4 – Analyze]

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
- Clothing — with size, fabric, gender
- Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules.

Analyze why polymorphism makes the cart checkout process flexible and extensible.

**OOP Concepts:** *Polymorphism | Inheritance | Method Overriding*

---

## 9. University Staff Hierarchy [BL4 – Analyze]

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure

Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

**OOP Concepts:** *Multilevel Inheritance | Access Modifiers | Data Hiding*

---

## 10. Smart Home Device Controller [BL4 – Analyze]

Analyze and design a SmartDevice superclass with subclasses: SmartLight,

SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior.

Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed)

Design the class hierarchy and explain the OOP principles applied at each level.

**OOP Concepts:** *Polymorphism* | *Encapsulation* | *Inheritance*

### RUBRICS FOR CALCULATION:

| Criteria                  | Weightage | Level 4 – Exemplary                                                   | Level 3 – Proficient                      | Level 2 – Developing         | Level 1 – Beginning             |
|---------------------------|-----------|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20%       | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30%       | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20%       | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10%       | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20%       | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |